Full Stack AI Chatbot Web Application

I want you to act as a senior full-stack software architect and engineer. Your goal is to build a production-ready AI chatbot web application with a clean, scalable architecture because I will be adding additional AI workflow nodes in the future.

Do not generate a quick prototype. Generate clean, modular, well-documented, production-quality code following best practices.

Tech Stack

Frontend

- React (latest version)
- Bootstrap 5 (no Tailwind CSS)
- React Router
- Axios
- React Query (TanStack Query) for server state management
- Context API for authentication and UI state
- Responsive design
- Modern ChatGPT-like interface

Backend

Use:

- Python
- FastAPI
- LangGraph
- Google Gemini API
- PostgreSQL
- Redis
- SQLAlchemy ORM
- Alembic
- Pydantic
- JWT Authentication
- Google OAuth 2.0
- Background workers using FastAPI BackgroundTasks (or Celery if appropriate)
- Docker-ready project structure

AI Architecture

The chatbot **must** use LangGraph.

Even though the graph initially contains only one node, I specifically want LangGraph because I will expand the workflow later.

Current graph:

User Input

↓

Gemini Model Node

↓

Return Response

Every request must pass through the LangGraph graph.

Do **not** bypass LangGraph.

Design the graph so additional nodes can easily be added later without major refactoring.

Future nodes may include:

- Memory
- RAG
- Tool calling
- SQL Agent
- Web Search
- Reflection
- Planner
- Multi-agent workflows

Gemini Integration

Use Google's Gemini API.

- Read the API key from environment variables.
- Never hardcode secrets.
- Create a model service abstraction so another LLM can easily replace Gemini later.

Authentication

Implement complete authentication using **Google OAuth 2.0**.

Workflow:

Landing Page

↓

Google Login

↓

Backend verifies Google token

↓

Create user if first login

↓

Issue JWT Access Token

↓

Issue Refresh Token

↓

Redirect to Chat

The frontend must never trust Google's token directly.

The backend must verify everything.

Login Persistence

This is extremely important.

After logging in, the user should remain logged in even if:

- the page refreshes
- the browser refreshes
- the browser closes
- the website is reopened

The session should remain valid for approximately **15 minutes**.

Use:

- JWT Access Token
- Refresh Token

- Secure HTTP-only cookies where appropriate

Implement automatic token refresh.

When both tokens expire, require the user to log in again.

Chat Interface

Create a professional ChatGPT-like interface.

Features:

- Left sidebar
- New Chat button
- Previous conversations
- Conversation titles
- Rename chat
- Delete chat
- Markdown rendering
- Syntax-highlighted code blocks
- Copy code button
- Auto scrolling
- Typing animation
- Loading indicator
- Responsive layout
- Streaming AI responses using WebSockets or Server-Sent Events (SSE)

Conversation Storage Architecture

This is one of the most important parts of the project.

I want Redis and PostgreSQL used together correctly.

PostgreSQL

PostgreSQL is the **source of truth**.

Every message must ultimately be stored permanently in PostgreSQL.

No conversation should ever exist only in Redis.

Redis

Redis should only be used as:

- active conversation cache

- session cache
- fast retrieval
- temporary in-memory storage
- performance optimization

Redis is **not** the permanent database.

Message Flow

When a user sends a message:

1. Receive the message.
2. Store the message in Redis immediately.
3. Asynchronously persist the message to PostgreSQL using a background task (within a few seconds at most).
4. Execute the LangGraph workflow.
5. Receive the Gemini response.
6. Store the response in Redis.
7. Asynchronously persist the response to PostgreSQL.
8. Return the streamed response to the frontend.

The persistence delay should only exist because it runs asynchronously.

It should **not** wait minutes before writing.

If Redis crashes, PostgreSQL should still contain all persisted conversations.

Conversation History

Every conversation should be permanently stored.

Users should be able to:

- view previous chats
- continue old chats
- rename chats
- delete chats
- search chats (optional)

Chat history must remain available after logging out and logging back in.

PostgreSQL Schema

Create normalized tables.

Users

- id
- google_id
- email
- name
- profile_picture
- created_at

Chats

- id
- user_id
- title
- created_at
- updated_at

Messages

- id
- chat_id
- role
- content
- timestamp

Indexes should be added where appropriate.

Use SQLAlchemy models and Alembic migrations.

Backend APIs

Design REST APIs.

Authentication

- Login
- Logout
- Refresh Token
- Current User

Chats

- Create Chat
- Get Chats
- Get Chat
- Rename Chat
- Delete Chat

Messages

- Send Message
- Stream Response
- Get Messages

Users

- Profile
- Update Profile

Frontend Pages

Create at minimum:

- Landing Page
- Login Page
- Chat Page
- Profile Page
- 404 Page

Project Structure

Backend

app/

api/

auth/

database/

models/

schemas/

services/

langgraph/

graphs/

nodes/

redis/

middleware/

config/

utils/

workers/

main.py

Frontend

src/

components/

pages/

layouts/

hooks/

context/

services/

routes/

utils/

assets/

Security

Implement production-level security.

Include:

- Google OAuth verification
- JWT authentication
- Refresh tokens
- Secure HTTP-only cookies where appropriate
- CORS configuration
- Input validation
- SQL injection protection
- XSS protection
- Environment variables

- Rate limiting

Never expose secrets.

Environment Variables

Create an example .env file.

Include:

DATABASE_URL

REDIS_URL

GEMINI_API_KEY

GOOGLE_CLIENT_ID

GOOGLE_CLIENT_SECRET

JWT_SECRET

JWT_REFRESH_SECRET

ACCESS_TOKEN_EXPIRE_MINUTES

REFRESH_TOKEN_EXPIRE_DAYS

Coding Standards

Follow:

- SOLID principles
- Clean Architecture
- Dependency Injection where appropriate
- Repository pattern for database access if beneficial
- Type hints
- Modular code
- Minimal but meaningful comments
- Consistent naming conventions

Avoid duplicated code.

Future Expandability

The project should already be structured for future LangGraph expansion.

Adding new nodes should require minimal changes.

Future nodes may include:

- RAG
- Memory
- Tool calling
- SQL Agent
- Web Search
- Reflection
- Planning
- Multi-Agent collaboration

Do not design anything that tightly couples the Gemini model to the application.

The AI layer should be easily replaceable.

Deliverables

Build the project incrementally.

For every phase:

1. Explain the architecture.
2. Generate the complete directory structure.
3. Generate complete production-ready code.
4. Explain every important file.
5. Explain how to configure PostgreSQL, Redis, Google OAuth, and Gemini.
6. Explain how to run the application.
7. Wait for approval before moving to the next phase.

Do not skip files.

Do not use placeholders unless absolutely necessary.

Generate complete, runnable, production-quality code.

The final application should allow a user to:

- Sign in with Google OAuth.
- Remain logged in for approximately 15 minutes using JWT + refresh tokens.
- Chat with Gemini through a LangGraph workflow.
- Stream AI responses.
- Store active conversations in Redis.

- Persist every conversation to PostgreSQL within a few seconds using asynchronous background tasks.
- View, continue, rename, and delete previous conversations.
- Scale easily as additional LangGraph nodes are introduced in the future.